Introducción a Git y GitHub

Control de versiones para desarrolladores

M.C. Edwin Enrique Pérez Rodríguez
Dr. Juan Carlos Cajas García

ENES Mérida

March 3, 2026

Contenido

1. ¿Qué es Git?
2. ¿Qué es GitHub?
3. Instalación de Git
4. Configuración de Git
5. Comandos principales de Git
6. Descargar repositorios de GitHub
7. Recursos adicionales

¿Qué es Git?

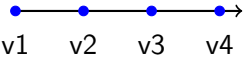
- **Sistema de control de versiones distribuido**
- Creado por Linus Torvalds en 2005
- Permite rastrear cambios en archivos y coordinar trabajo entre múltiples personas
- Características principales:
  - Rápido y eficiente
  - Distribuido (cada desarrollador tiene una copia completa del historial)
  - Soporte para desarrollo no lineal (ramas)
  - Gratuito y de código abierto

¿Por qué usar Git?

Ventajas:

- Historial completo de cambios
- Trabajo colaborativo
- Experimentación segura (ramas)
- Reversión de cambios
- Respaldo distribuido

Línea de tiempo



Los tres estados de Git

Git tiene tres estados principales en los que pueden estar tus archivos:

1. **Modified (Modificado):** Has cambiado el archivo pero aún no lo has confirmado
2. **Staged (Preparado):** Has marcado el archivo modificado para ir en tu próximo commit
3. **Committed (Confirmado):** Los datos están almacenados de forma segura en tu repositorio local

Esto nos lleva a las tres secciones principales de un proyecto Git:

- Working Directory (Directorio de trabajo)
- Staging Area (Área de preparación)
- Git Directory (Repositorio)

¿Qué es GitHub?

- **Plataforma de alojamiento de código**
- Basada en Git
- Funcionalidades adicionales:
  - Repositorios remotos
  - Colaboración (Pull Requests, Issues)
  - Revisión de código
  - GitHub Actions (CI/CD)
  - GitHub Pages (hosting de sitios web)
  - Control de acceso y permisos
- Alternativas: GitLab, Bitbucket, Gitea

Git vs GitHub

Git	**GitHub**
Sistema de control de versiones	Servicio de hosting
Software local	Plataforma web
Línea de comandos	Interfaz gráfica
Gratuito y open source	Gratuito con planes de pago
Funciona sin internet	Requiere internet

Instalación en Windows

1. Descargar desde: `https://git-scm.com/download/win`
2. Ejecutar el instalador
3. Opciones recomendadas:
   - Use Git from the command line.
   - Use the OpenSSL library.
   - Checkout Windows-style, commit Unix-style line endings.
4. Verificar instalación:

```
git --version
```

Listing 1: Verificar versión de Git

Salida esperada: `git version 2.x.x`

Instalación en Linux

Ubuntu/Debian:

```
sudo apt update
sudo apt install git
```

Verificar instalación:

```
git --version
```

Configuración inicial

Configurar nombre de usuario:

```
1 git config --global user.name "Tu Nombre"
```

Configurar correo electrónico:

```
1 git config --global user.email "tu@email.com"
```

Configurar editor por defecto (opcional):

```
1 git config --global core.editor "vim"
2 git config --global core.editor "code --wait"
3 git config --global core.editor "nano"
```

Ver configuración

Ver toda la configuración:

```
git config --list
```

Ver configuración específica:

```
git config user.name
git config user.email
```

Ver ubicación de archivos de configuración:

```
git config --list --show-origin
```

Crear un repositorio

Inicializar un nuevo repositorio:

```
mkdir mi-proyecto
cd mi-proyecto
git init              "Iniciar rastreo de cambios."
```

Verificar estado:

```
git status       "Ver cambios realizados hasta el momento."
```

Agregar y confirmar cambios

Agregar archivos al área de staging:

```
git add archivo.txt "Agregar un archivo en especifico"
git add .           "Agregar todos los archivos que fueron modificados."
git add *.py        "Agregar los cambios realizados en todos los archivos"
                    "que terminen en .py"
```

Confirmar cambios (commit):

```
git commit -m "Mensaje descriptivo del cambio"
git commit -am "Mensaje del commit (para nuevos cambios hechos a un commit"
               "reciente)."
```

Ver historial

Ver historial de commits:

```
git log              "Enlista todos los commits realizados."
git log --oneline    "Muestra el commit mas reciente."
git log -n 5         "Enlista los ultimos 5 commits mas recientes."
```

Ver diferencias:

```
git diff archivo.extension          "Muestra los cambios que NO estan en el"
                                    "staging area. Cambios que aun no has"
                                    "agregado con git add"

git diff --staged archivo.extension "Muestra los cambios que SI estan en el"
                                    "staging area. Cambios ya agregados"
                                    "con git add."
```

Trabajar con ramas

La Rama Main

La rama main es la rama principal del proyecto. Es como el "tronco" del árbol:

- Representa el código que está en producción o listo para producción
- Es la rama por defecto cuando clonas o creas un repositorio
- Generalmente contiene código probado y funcional
- Es donde finalmente se integran todos los cambios aprobados

Buena práctica: Nunca trabajes directamente en main. Protégela para evitar errores accidentales.

Subramas (Feature Branches / Ramas de trabajo) Las subramas son ramas temporales que se crean desde main para trabajar en:

- Nuevas funcionalidades
- Correcciones de bugs
- Experimentos
- Desarrollo individual

Crear y cambiar ramas:

```
git branch  "Enlista todas las ramas del repositorio."
git branch nueva-funcionalidad  "Crea una nueva rama."
git checkout nueva-funcionalidad    "Cambiar a la rama llamada:
                                    "nueva-funcionalidad."
```

Deshacer cambios

Deshacer cambios del staging area:

```
1 git restore archivo.extension
```

Clonar un repositorio

Clonar repositorio completo:

```
1 git clone <url-del-repositorio>
2 git clone https://github.com/usuario/repositorio.git
3
4 "Clonar repositorio a carpeta especifica."
5 git clone https://github.com/usuario/repo.git mi-carpeta
```

Clonar rama específica:

```
1 git clone -b nombre-rama https://github.com/usuario/repo.git
```

Trabajar con repositorios remotos

Ver repositorios remotos:

```
git remote -v
```

Agregar repositorio remoto:

```
git remote add origin https://github.com/usuario/repo.git
```

Subir cambios (push):

```
git push -u origin main
git push
```

Descargar cambios (pull):

```
git pull origin main
```

Flujo de trabajo completo

```
1  git clone https://github.com/usuario/proyecto.git
2  cd proyecto
3
4  "Si se trabaja en otra rama que no se la main"
5  git checkout -b mi-funcionalidad
6
7  git add .
8  git commit -m "Agregar nueva funcionalidad"
9
10 git push -u origin mi-funcionalidad
11
12 "Bajar cambios hechos por otros usuarios"
13 git pull origin main
```

Recursos para aprender más

- **Documentación oficial:**
  - Git: `https://git-scm.com/doc`
  - GitHub: `https://docs.github.com`
- **Tutoriales interactivos:**
  - Learn Git Branching: `learngitbranching.js.org`
  - GitHub Skills: `skills.github.com`
- **Libros gratuitos:**
  - Pro Git: `git-scm.com/book`

¡Gracias!

¿Preguntas?